

Consumer credit scoring

Machine Learning Project 2025

Juan Carlos PAJARES, Jordy SALTOS

Business objective

- Reduce financial losses by distinguishing borrowers who are likely to repay (Good) from those who present high default risk (Bad).

Problem Framing

- Learning type : Supervised learning
- Task type : Binary classification: predict probability that an applicant will stay « Good » in the future
- Learning setup : Batch learning (train on historical data), offline

Our performance measure of choice : F1-score

- Our goal is to maximise precision and recall of the « Good » category,
- F1-score is a combination of the two :

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Current solutions from the literature

Primarily implemented models	Widely used DL models
AdaBoost	ANN
SVM	MLP(Deep Multi-Layer Perceptron)
Tree-related	
k-NN	
Bagging	

Results from a 2022 study on two benchmark datasets from the UCI Machine Learning repository

Current solutions from the literature

	German dataset	Australian dataset
Highest AUC	Bagging (Bootstrap Aggregating)	ANN
Highest ACC	RF	ELM (Extreme Learning Machine)

Results from a 2022 study on two benchmarks datasets from the UCI Machine Learning repository



Notebooks Expert
2,614 of 54,970

sayamkumar

Sayam Kumar

📍 Toronto, Ontario, Canada

📅 Joined 4 years ago · last seen 2 days ago

Current solution for our dataset on Kaggle

	precision	recall	f1-score
0	0.71	0.79	0.74
1	0.70	0.70	0.70
<u>2</u>	0.62	0.55	<u>0.58</u>
accuracy	0.68		
macro avg	0.67	0.68	0.67
weighted avg	0.67	0.68	0.67

Report for the LogR model

	precision	recall	f1-score
0	0.77	0.85	0.81
1	0.80	0.74	0.77
<u>2</u>	0.73	0.70	<u>0.72</u>
accuracy	0.77		
macro avg	0.77	0.77	0.77
weighted avg	0.77	0.77	0.77

Report for GradientBoostingCl

	precision	recall	f1-score
0	0.85	0.92	0.88
1	0.86	0.86	0.86
<u>2</u>	0.83	0.76	<u>0.79</u>
accuracy			0.85
macro avg	0.85	0.85	0.85
weighted avg	0.85	0.85	0.85

Report for the RF model

	precision	recall	f1-score
0	0.83	0.89	0.86
1	0.83	0.84	0.84
<u>2</u>	0.80	0.73	<u>0.76</u>
accuracy			0.82
macro avg	0.82	0.82	0.82
weighted avg	0.82	0.82	0.82

Report for Bagging

Data Preprocessing

Dataset Overview & Initial Issues

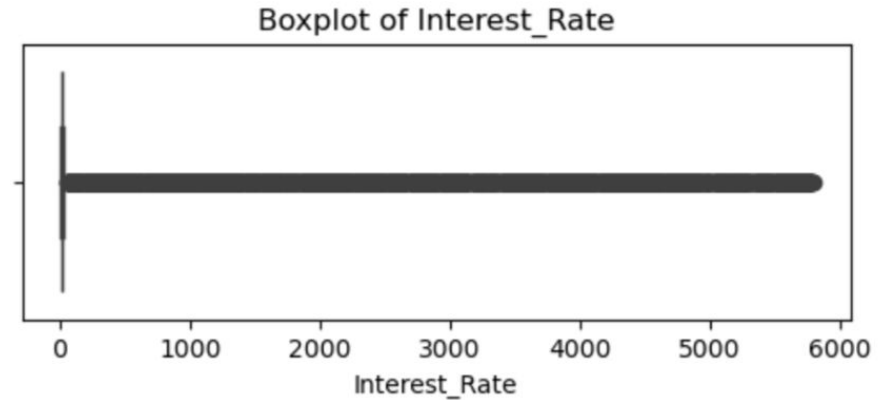
- Loaded dataset ($\approx 100,000$ observations, 28 variables)
- First inspection showed inconsistent entries and missing values
- Several variables clearly irrelevant for prediction
 - Removed: ID, SSN, Name, Credit_History_Age, Type_of_Loan, etc.

Fixing Inconsistent Categories

- Detected anomalies in categorical fields
 - Example: “_____” replaced by “No Job”
 - “!@9#%8” → replaced by “Low_spent_Small_value_payments”
- Standardized Credit_Mix and Credit_Score categories (“Standard” & “Poor” → merged into “Bad”).
 - Later target variable Credit_Score will be recoded to binary (Bad = 0, Good = 1)
- Label encoded categorical variables for modeling

Outlier Analysis (Before Cleaning)

- Built individual boxplots for every numerical feature
- Some variables exhibited extreme values (Age, Income, Debt...)
- These outliers distorted distributions and could harm model performance



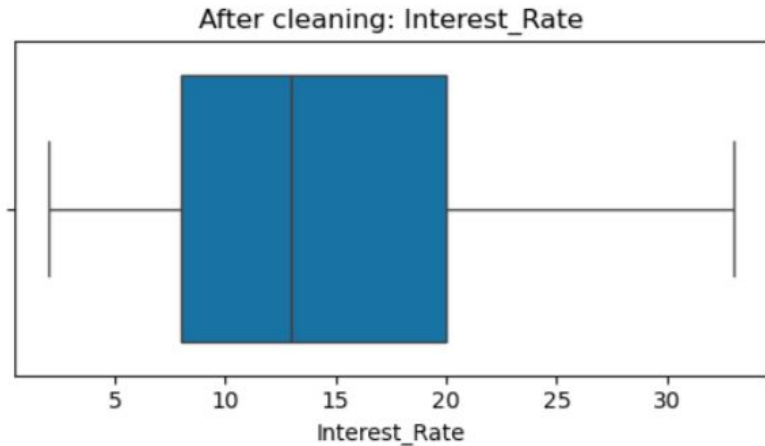
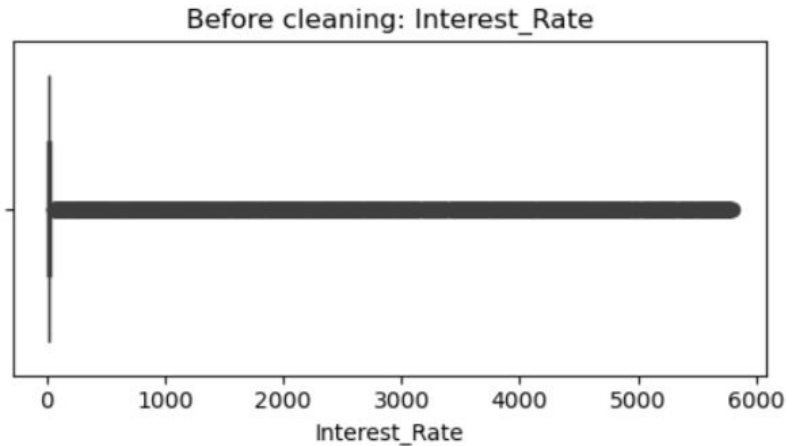
Outlier Removal Strategy

- Replaced values outside [5th, 95th percentile] with NaN
 - Removes extreme anomalies without arbitrary cutoffs

```
def replace_outliers_with_NaN(df, columns, lower_percentile=5, upper_percentile=95):  
    """  
    Replace outliers with NaN using percentiles  
    """  
    df_clean = df.copy()  
    reporte = {}  
  
    for column in columns:  
        # Calculate limits  
        limite_inferior = df_clean[column].quantile(lower_percentile/100)  
        limite_superior = df_clean[column].quantile(upper_percentile/100)  
  
        # Identify outliers  
        outliers_mask = (df_clean[column] < limite_inferior) | (df_clean[column] > limite_superior)  
        outliers_indices = df_clean[outliers_mask].index.tolist()  
        outliers_valores = df_clean.loc[outliers_indices, column].tolist()  
  
        # Replace outliers with NaN  
        n_outliers = outliers_mask.sum()  
        df_clean.loc[outliers_mask, column] = np.nan  
  
    return df_clean  
  
# Use percentiles  
df_percentiles = replace_outliers_with_NaN(df_train, numeric_cols)
```

Outlier Removal Strategy

- Visualized before/after boxplots to confirm distribution improvement
- Identified which columns gained NaN values



Block Imputation by Customer_ID

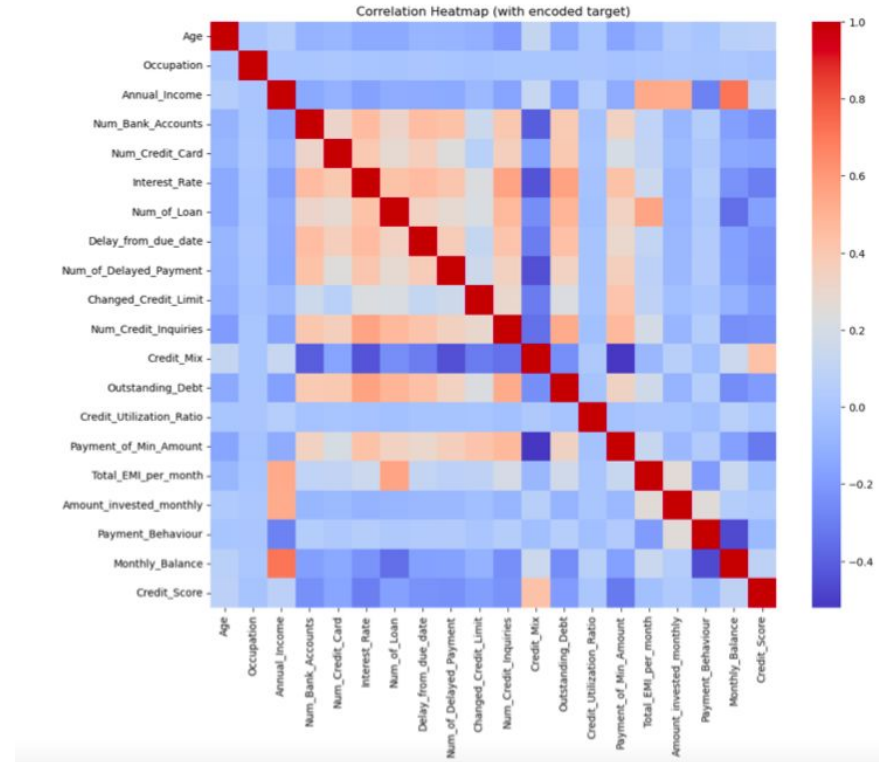
- Each customer appears 8 times (8 months)
- Missing values replaced by majority value within the same customer
- Ensures consistency across time for each client

```
def fix_blocks(df):  
    df = df.copy()  
    n = len(df)  
    for elem in df["Customer_ID"].unique():  
        block=df[df["Customer_ID"] == elem]  
  
        for col in columns_with_NaN:  
            values = block[col].tolist()  
  
            # If all values in the block are identical : nothing to change  
            if len(set(values)) == 1:  
                continue  
  
            # If values differ : replace with the majority value  
            try:  
                majority_value = mode(values)  
            except StatisticsError:  
                # If there is a tie : use the first value in the block  
                majority_value = values[0]  
  
            # Replace all 8 rows with the majority value  
            df.loc[(df["Customer_ID"] == elem) & (df[col].isna()), col] = majority_value  
    return df
```

```
DatasetFinal = fix_blocks(df_percentiles)
```

Correlation Study (Numerical Features)

- Computed Pearson correlation matrix
- Very few strong correlations detected (>0.6 only 1 pair)
- No strong linear relationship between features and target
 - Credit_Score is not linearly driven by any single variable



Why Create New Variables?

- Correlation analysis showed weak linear relationships
- Boxplots revealed overlapping distributions for Good vs Bad clients
- Credit behavior is often explained by relationships between variables, not raw variables alone
 - Debt relative to income
 - Delays relative to number of accounts
- To capture these nonlinear relationships, we create engineered features

Creation of New Predictive Variables

- Total number of accounts = Bank + Credit cards
- Debt per account = $\text{Outstanding_Debt} / \text{Total accounts}$
- Debt-to-income ratio = $\text{Outstanding_Debt} / \text{Annual_Income}$
- Delayed payments per account = $\text{Delays} / \text{Total accounts}$

Choosing the Most Predictive Features

- Mutual Information (MI) detects nonlinear relationships
- Works for both numerical and categorical variables
- Ranked features by MI score
- Removed features below 2% MI threshold

Model Training

Data Splitting and Normalization

- Applied StandardScaler to normalize numerical features
 - improves logistic regression and gradient boosting performance
- Split data into:
 - 80% training
 - 20% test
- Observed initial class imbalance in y_train

```
scaler = StandardScaler()  
X = scaler.fit_transform(X_df)
```

Balancing Classes with SMOTE

- Credit scoring datasets often contain more “Bad” clients than “Good”
- Applied SMOTE (Synthetic Minority Oversampling Technique)
- Balanced both training and test sets to
 - Avoids bias toward majority class

Models Considered for Evaluation

Tested four supervised classification models:

- Logistic Regression (baseline linear model)
- Random Forest (bag of decision trees)
- Bagging Classifier (ensemble of bootstrapped models)
- Gradient Boosting (sequential tree-based boosting)

Wrapped each model in a scikit-learn Pipeline

Hyperparameter Optimization

- Performed GridSearchCV with 5-fold cross-validation
- Evaluation metric: Weighted F1-score
- Result Summary:
 - Logistic Regression : $F1 \approx 0.78$
 - Random Forest : $F1 \approx 0.86$
 - Gradient Boosting : $F1 \approx 0.85$
 - Bagging Classifier : best CV score ≈ 0.879

Final Training on Full Balanced Dataset

- Re-created pipelines using best hyperparameters
- Retrained models on full SMOTE-balanced training set
- Evaluated on SMOTE-balanced test set

```
bagging
```

```
Best parameters: {'model__bootstrap': False, 'model__max_features': 0.5, 'model__max_samples': 1.0, 'model__n_estimators': 200}
```

```
CV score: 0.8760
```

```
Fitting 5 folds for each of 81 candidates, totalling 405 fits
```

```
gradient boost
```

Performance evaluation

Test Performance Comparison (evaluating Good Clients)

=====

EVALUATION: LOGISTIC

=====

METRICS FOR GOOD CLIENTS:

F1-Score:	0.7848
Precision:	0.7886
Recall:	0.7810
Accuracy:	0.7858
AUC-ROC:	0.8463

=====

EVALUATION: GRADIENT_BOOST

=====

METRICS FOR GOOD CLIENTS:

F1-Score:	0.9397
Precision:	0.9477
Recall:	0.9319
Accuracy:	0.9402
AUC-ROC:	0.9855

=====

EVALUATION: BAGGING

=====

METRICS FOR GOOD CLIENTS:

F1-Score:	0.9466
Precision:	0.9561
Recall:	0.9373
Accuracy:	0.9471
AUC-ROC:	0.9828



=====

EVALUATION: RANDOM_FOREST

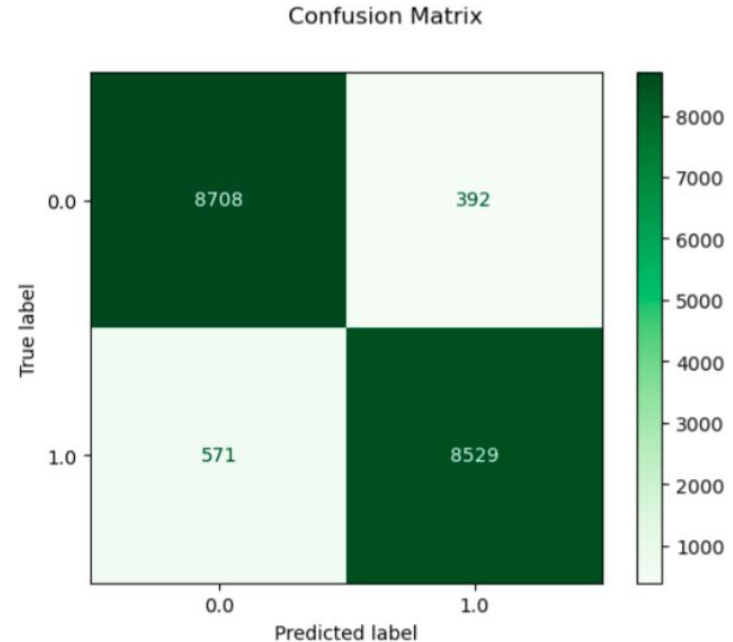
=====

METRICS FOR GOOD CLIENTS:

F1-Score:	0.9152
Precision:	0.9538
Recall:	0.8797
Accuracy:	0.9185
AUC-ROC:	0.9666

Confusion Matrix (Bagging Classifier)

Bagging maintains a very low False Positive Rate, making it suitable for credit-risk applications.



Computational Performance

- Measured training time of Bagging model: ≈ 199 seconds
- Measured prediction time per bootstrap sample: ≈ 1.00 seconds
- Computation cost acceptable for production scoring.

Confidence intervals using Bootstrap

- Bootstrapped test set 100 times. Computed CI for:
 - Accuracy
 - Precision
 - Recall
 - F1-score
 - Specificity
 - False Positive Rate
 - Prediction time
- 95% CI results show low variance, confirming model stability

Model	Bagging Classifier
Training Time (s)	189.46
Accuracy	0.9477 $\pm$ 0.0032
Precision	0.9587 $\pm$ 0.0037
Recall	0.9355 $\pm$ 0.0052
F1-Score	0.9470 $\pm$ 0.0033
Specificity	0.9598 $\pm$ 0.0037
FPR	0.0402 $\pm$ 0.0037
Pred Time (s)	0.9164 $\pm$ 0.1721

Conclusion & Selected Model

- Bagging classifier provides:
 - best F1-score
 - lowest false positives
 - high model stability
 - strong predictive performance
- Suitable for credit scoring due to:
 - strong separation between Good/Bad clients
 - minimizes dangerous approvals

Bibliography

Shi, S., Tse, R., Luo, W., D'Addona, S., & Pau, G. (2022). *Machine learning-driven credit risk: A systemic review*. *Neural Computing and Applications*, 34, 14327–14339.

<https://doi.org/10.1007/s00521-022-07472-2>